

12.6.3 质量属性场景描述

为了精确描述软件系统的质量属性，通常采用质量属性场景（Quality Attribute Scenario）作为描述质量属性的手段。质量属性场景是一个具体的质量属性需求，是利益相关者与系统的交互的简短陈述。

质量属性场景是一种面向特定质量属性的需求。它由6部分组成：

刺激源（Source）：指某个生成该刺激的实体（人、计算机系统或者任何其他刺激器）。

刺激（Stimulus）：该刺激是当刺激到达系统时需要考虑的条件。

环境（Environment）：该刺激在某些条件下发生。当激励/刺激产生时，系统可能处于过载或者运行状态，也可能是其他情况。

制品（Artifact）：某个制品被激励。这可能是整个系统，也可能是系统的一部分。

响应（Response）：该响应是在激励到达后所采取的行动。

响应度量（Measurement）：当响应发生时，应当能以某种方式对其进行度量，以对需求进行测试。

质量属性场景主要关注可用性、可修改性、性能、可测试性、易用性和安全性等6类质量属性，下面分别列表进行介绍。

1. 可用性质量属性场景

可用性质量属性场景所关注的方面包括系统故障发生的频率、出现故障时会发生什么情况、允许系统有多长时间正常运行、什么时候可以安全地出现故障、如何防止故障的发生以及发生故障时要求进行哪种通知。表12-1给出了详细描述。

表 12-1 可用性质量属性场景描述

场景要素	可能的情况
刺激源	系统内部、系统外部
刺激	疏忽、错误、崩溃、时间
环境	正常操作、降级模式
制品	系统处理器、通信信道、持久存储器、进程
响应	系统应该检测事件，并进行如下一项或多项活动： ● 将其记录下来通知适当的各方，包括用户和其他系统 ● 根据已定义的规则禁止导致错误或故障的事件源 ● 在一段预先指定的时间间隔内不可用，时间间隔取决于系统的关键程度在正常或降级模式下运行
响应度量	系统必须可用的时间间隔、可用时间、系统可以在降级模式下运行的时间间隔、故障修复时间

2. 可修改性质量属性场景

可修改性质量属性场景主要关注系统在改变功能、质量属性时需要付出的成本和难度，可

修改性质量属性场景可能发生在系统设计、编译、构建、运行等多种情况和环境下，表 12-2 给出了详细描述。

表 12-2 可修改性质量属性场景描述

场景要素	可能的情况
刺激源	最终用户、开发人员、系统管理员
刺激	希望增加、删除、修改、改变功能、质量属性、容量等
环境	系统设计时、编译时、构建时、运行时
制品	系统用户界面、平台、环境或与目标系统交互的系统
响应	查找架构中需要修改的位置，进行修改且不会影响其他功能，对所做的修改进行测试，部署所做的修改
响应度量	根据所影响元素的数量和修改需求难度等，度量修改该元素的人力成本、修改所需的时间、修改的资金成本

3. 性能质量属性场景

性能质量属性场景主要关注系统的响应速度，可以通过效率、响应时间、吞吐量、负载来客观评价性能的好坏，表 12-3 给出了详细描述。

表 12-3 性能质量属性场景描述

场景要素	可能的情况
刺激源	用户请求，其他系统触发等
刺激	定期事件到达；随机事件到达；偶然事件到达
环境	正常模式；超载（overload）模式
制品	系统
响应	处理刺激；改变服务级别
响应度量	等待时间、期限、吞吐量、抖动、缺失率、数据丢失率

4. 可测试性质量属性场景

可测试性质量属性场景主要关注系统测试过程中的效率，发现系统缺陷或故障的难易程度等，表 12-4 给出了详细描述。

表 12-4 可测试性质量属性场景描述

场景要素	可能的情况
刺激源	开发人员、增量开发人员、系统验证人员、客户验收测试人员、系统用户
刺激	已完成的分析、架构、设计、类和子系统集成；所交付的系统
环境	设计时、开发时、编译时、部署时
制品	设计、代码段、完整的应用
响应	提供对状态值的访问、提供所计算的值、准备测试环境
响应度量	已执行的可执行语句的百分比、如果存在缺陷出现故障的概率、执行测试的时间、测试中最长依赖的长度、准备测试环境的时间

5. 易用性质量属性场景

易用性质量属性场景主要关注用户在使用系统时的容易程度，包括系统的学习曲线、完成操作的效率、对系统使用过程的满意程度等，表 12-5 给出了详细描述。

表 12-5 易用性质量属性场景描述

场景要素	可能的情况
刺激源	最终用户
刺激	想要学习系统特性、有效使用系统、使错误的影响最低、适配系统、对系统满意
环境	系统运行时或配置时
制品	系统
响应	(1) 系统提供以下一个或多个响应来支持“学习系统特性”： 帮助系统与环境联系紧密；界面为用户所熟悉；在不熟悉的环境中，界面是可以使用的 (2) 系统提供以下一个或多个响应来支持“有效使用系统”： 数据和（或）命令的聚合；已输入的数据和（或）命令的重用；支持在界面中的有效导航；具有一致操作的不同视图；全面搜索；多个同时进行的活动 (3) 系统提供以下一个或多个响应来“使错误的影响最低”： 撤销；取消；从系统故障中恢复；识别并纠正用户错误；检索忘记的密码；验证系统资源 (4) 系统提供以下一个或多个响应来“适配系统”： 定制能力；国际化 (5) 系统提供以下一个或多个响应来使客户“对系统满意”： 显示系统状态；与客户的节奏合拍
响应度量	任务时间、错误数量、解决问题的数量、用户满意度、用户知识的获得、成功操作在总操作中所占的比例、损失的时间 / 丢失的数据量

6. 安全性质量属性场景

安全性质量属性场景主要关注系统在安全性方面的要素，衡量系统在向合法用户提供服务的同时，阻止非授权用户使用的能力，表 12-6 给出了详细描述。

表 12-6 安全性质量属性场景描述

场景要素	可能的情况
刺激源	正确识别、非正确识别或身份未知的个人或系统（来自内部或外部）；经过了授权 / 未授权访问了有限的资源或大量资源
刺激	试图显示数据、改变 / 删除数据、访问系统服务、降低系统服务的可用性
环境	在线或离线、联网或断网、连接有防火墙或者直接连到了网络
制品	系统服务、系统中的数据
响应	对用户身份进行认证；隐藏用户的身份；阻止对数据或服务的访问；允许访问数据或服务；授予或收回对访问数据或服务的许可；根据身份记录访问 / 修改或试图访问 / 修改数据 / 服务；以一种不可读的格式存储数据；识别无法解释的对服务的高需求；通知用户或另外一个系统，并限制服务的可用性
响应度量	用成功的概率表示，避开安全防范措施所需要的时间 / 努力 / 资源；检测到攻击的可能性；确定攻击或访问 / 修改数据或服务的个人的可能性；在拒绝服务攻击的情况下仍然获得服务的百分比；恢复数据 / 服务；被破坏的数据 / 服务和 / 或被拒绝的合法访问的范围

第 13 章 系统设计

系统设计是系统分析的延伸与拓展。系统分析阶段解决“做什么”的问题，而系统设计阶段解决“怎么做”的问题。同时，它也是系统实施的基础，为系统实施工作做好铺垫。合理的系统设计方案既可以保证系统的质量，也可以提高开发效率，确保系统实施工作的顺利进行。

系统设计阶段又称为物理设计阶段，它是信息系统开发过程中一个非常重要的阶段。其任务是根据系统规格说明书中规定的功能要求，考虑实际条件，具体设计实现逻辑模型的技术方案，也就是设计新系统的物理模型，为下一阶段的系统实施工作奠定基础。

13.1 系统设计概述

系统设计的目标是根据系统分析的结果，完成系统的构建过程。其主要目的是绘制系统的蓝图，权衡和比较各种技术和实施方法的利弊，合理分配各种资源，构建新系统的详细设计方案和相关模型，指导系统实施工作的顺利开展。

系统设计的主要内容包括概要设计和详细设计。概要设计又称为系统总体结构设计，它是系统开发过程中很关键的一步，其主要任务是将系统的功能需求分配给软件模块，确定每个模块的功能和调用关系，形成软件的模块结构图，即系统结构图。在概要设计中，将系统开发的总任务分解成许多个基本的、具体的任务，而为每个具体任务选择适当的技术手段和处理方法的过程称为详细设计。根据任务的不同，详细设计又可分为多种，例如，网络设计、代码设计、输入/输出设计、处理流程设计、数据存储设计、用户界面设计、安全性和可靠性设计等。

1. 网络设计

网络设计的主要任务是，根据系统的要求选择网络结构，按照系统结构的划分，安排网络 and 设备的分布，然后根据物理位置考虑网络布线和设备的部署，还要根据实际业务的要求划定各网络节点的权限、级别和管理方式等，选择相应的系统软件和管理软件。有关网络设计的详细知识见 4.4.2 小节。

2. 代码设计

代码是用数字或字符来表示各种客观实体。在系统开发过程中，进行代码设计的主要目的是确保代码的唯一化、规范化和系统化。在进行代码设计时，首先需要考虑系统的编码问题，编码问题的关键在于分类，有了科学的分类方式后，系统要建立编码规范就相对容易。准确的分类是工作标准化、系列化、合理化的基础和保证，目前最常用的分类方法概括起来有两种，一种是线分类法，另一种是面分类法。线分类法是目前使用最多的一种方法，尤其是在手工处

理的情况下几乎成了唯一的方法，其基本原理是：首先给定母项，母项下分若干子项，子项又可以分为更小的子项，最后落实到具体对象，分类的结果造成了一层套一层的线性关系。面分类法与线分类法不同，主要从面角度来考虑分类。

编码是分类问题的一种形式化描述，如果分类问题解决得较好，编码问题就变成了一个简单的用什么样的字符来表示的问题。目前，常用的编码方式包括顺序码（例如，用001表示北京、002表示长沙）、数字码（例如，用纯数字来表示居民身份证号码）、字符码（例如，汉语拼音和英文等）和混合码（以数字和字符混合形式编码）。

在进行代码设计时，首先需要确定编码对象，考察是否有标准代码（例如，国际标准、国家标准、部门标准或行业标准等），如果有相应的标准代码，则应该遵循这些标准代码；然后，需要确定代码的种类与类型，考虑代码的检错功能；最后，编写代码表。代码设计是一个科学管理的问题，应遵循唯一性、合理性、可扩展性、简单性、适用性、规范性和系统性等原则，设计出一个好的代码方案对于系统的开发工作来说是一件极为有利的事情。

3. 输入/输出设计

输入设计的目的是确保向系统输入数据的完整性、正确性和一致性，其主要内容包括确定输入数据内容、输入方式设计、输入格式设计和检验方式设计。输出设计的目的是确保系统输出数据的完整性、正确性和一致性，其主要内容包括确定输出的内容、选择输出设备与介质，以及确定输出格式等。

输入设计需要遵循以下原则：

（1）输入数据最少原则。在满足需求的前提下尽量提供较少的数据输入，数据的输入量越少，出错的概率越低，花费的时间也越少。

（2）简单性原则。输入过程应尽量简单，如以性别、出生日期等数据设计作为选择项。这样做一方面方便用户的使用，节省输入时间；另一方面可以降低出错的可能性。

（3）尽早验证原则。对输入数据的检验尽量接近数据的输入点，及时发现输入中存在的错误，以便能够尽早进行改正。

（4）少转换原则。输入数据尽量采用原始的数据格式，避免在数据转换过程中发生错误。

系统输出一般包括中间输出和最终输出，用户关心的是系统的最终输出。最常用的最终输出方式有两种，一种是报表输出，另一种是图形输出。一般来说，对于普通用户或具体数据的管理者和查阅者，应该以报表方式给出详细的数据记录；对于企业的高层领导和宏观或综合管理部门，则应该使用图形方式给出比例或综合发展趋势的信息，可以通过曲线图、柱状图、饼状图等图形方式来呈现。

4. 处理流程设计

处理流程设计是系统详细设计的重要组成部分，它的主要目的是确定各个系统模块的内部结构，即内部执行过程，包括局部数据组织和控制流，以及每个具体加工过程和实施细节。有关处理流程设计的详细知识将在13.2节中介绍。

5. 数据存储设计

数据存储设计主要是根据数据处理要求、处理方式、存储的信息量大小、数据使用的频率和所能提供的设备条件等，选择数据存储的方式、存储介质、数据组织方式和记录格式，并估算数据的容量。一个好的数据存储设计应该充分体现系统的业务流程，充分满足组织的各级管理要求。同时，还应该使得后续的系统开发工作方便、快捷，系统开销小，且易于管理和维护。

信息系统的主要目标是通过大量的数据获得管理所需要的信息，为了实现该目标，必须存储和管理大量的数据，因此，设计并建立一个良好的数据组织结构和数据库，使整个系统都可以迅速、方便、准确地存取和管理所需的数据，是衡量信息系统开发工作好坏的主要指标之一。数据存储设计直接影响到数据的存取效率、系统的实现效率和运行效率。数据存储设计主要包括两方面的工作：一是数据的统筹安排，例如，系统中设计多少个文件、数据文件如何分布、哪些数据是共享的、哪些是非共享的、哪些数据项应存放在一个文件中；二是文件的数据结构设计，目前，大部分应用系统都使用关系数据库来存储数据，因此，数据结构设计的重点是关系数据库的设计，主要过程包括数据库的概念设计、逻辑设计和物理设计。有关数据库设计的详细知识见 5.4 节。

6. 用户界面设计

界面是系统与用户交互的最直接的层面，界面的好坏决定用户对系统的第一印象，而设计优良的界面能够引导用户自己完成相应的操作，起到向导的作用。同时，界面如同人的面孔，具有吸引用户的直接优势，设计合理的界面能给用户带来轻松愉悦的感受和成功的感觉。相反，由于界面设计的失败，让用户有挫败感，再实用强大的功能都可能在用户的畏惧与放弃中付诸东流。通常情况下，良好的用户界面设计需要遵循以下基本原则：

(1) 置于用户控制之下。在定义人机交互方式时，不强迫用户采用不必要的或者不情愿的方式来进行操作，允许交互的中断和撤销。当用户操作技能等级提高时，可以实现流水化的交互方式，允许用户定制交互方式，以便使用户界面与内部技术细节隔离，允许用户和出现在屏幕上的对象直接进行交互。

(2) 减轻用户的记忆负担。尽量减轻对用户记忆的要求，创建有意义的默认设置，定义一些符合用户直觉的访问途径，适当定义一些快捷方式，界面的视觉布局应该尽量与真实世界保持一致，并能够以不断扩展的方式呈现信息。用户可以快速学习并使用系统，尽量提供“傻瓜式”的操作界面，方便用户使用。界面中各个元素的名称应该易懂，用词准确，避免模棱两可的字眼，使用户能够做到“望文知意”，理想的情况是用户不用查阅帮助，就能知道该界面元素的功能，并正确地进行相关操作。

(3) 保持界面一致性。用户应以一致的方式提供或获取信息，所有可视信息的组织需要按照统一的设计标准，在系列化的应用软件中需要保持一致性，用户已经很熟悉的一些界面交互模型不到万不得已时，不要随意进行修改。需要确保用户界面操作和使用的一致性，例如，所有窗口按钮的位置要一致、提示信息和界面元素的命名要一致、界面颜色和风格要一致等。用户界面的一致性可以使用户能够统一地对待系统的各个不同的功能界面，以及系列化的系统，